

I THREAD IN JAVA

TDP QUARTA



Programma quinta tdp

Programmazione di rete in Java

- Richiami sull'architettura della rete Internet:
 - Stack TCP/IP e funzione dei livelli (*)
 - UDP e TCP (*)
 - Modello client/server (*)
 - Modello peer-to-peer
- Realizzazione di applicazioni di rete in Java secondo il modello client/server
 - Applicazioni di rete basate su protocollo TCP (*)
 - Server multi-thread
 - Applicazioni di rete basate su protocollo UDP

Programma quinta tdp

XLM e web services

- **Sintassi e utilizzi del linguaggio XML ***
- **Definizione di schemi in linguaggio “XML Schema” ***
- Linguaggio XSL: fogli di stile per documenti XML
- Linguaggio XPath
- **Elaborazione di file XML in Java con l'API DOM ***
- **Web-service di tipo REST ***
- Web-service basati su SOAP e WSDL

Programma quinta tdp

Programmazione di applicazioni per dispositivi mobili

- Il sistema operativo Android: architettura, modello di sicurezza, componenti base di un'app
- Realizzazione di app Android:
 - Struttura base di un'app, tipi di risorse *
 - Activity e Intent *
 - Processi in background (AsyncTask) *
 - Broadcast Receivers
 - estensione della classe View
- Strumenti di sviluppo per Android
 - Creazione e gestione di un progetto in Android Studio
 - debug e release di una APP con Android Studio

Thread a livello utente

Java supporta i Thread a livello utente tramite:

- La classe **Thread**
- L'interfaccia **Runnable**
- La parola **synchronized**
- La classe **Semaphore**

Utilizzo Thread – primo metodo

Utilizzo di “extends Thread” (estende la classe Thread)

Start avvia il thread, esegue il costruttore e avvia il metodo run() che è il corpo del thread. Le istruzioni dopo test.start() sono eseguite immediatamente

```
package provathread;

public class ProvaThread extends Thread {
    public ProvaThread () {
    }
    @Override
    public void run() {
        System.out.println(x: "Hello world");
    }

    public static void main(String[] args) {
        ProvaThread test = new ProvaThread ();
        test.start();           //avvio il thread
    }
}
```

ESEMPIO

```
package thread1;

public class Thread1 {
    public static void main(String[] args)
    {
        Counter[] m = new Counter[10];
        for(int i = 0; i < 10; i++)
        {
            m[i] = new Counter();
            m[i].setName(Integer.toString(i));
            m[i].start();
        }
        System.out.println("Fine");
    }
}
```

```
package thread1;

public class Counter extends Thread {
    @Override
    public void run()
    {
        System.out.println("Thread ID " + this.getName());
        for(int i = 0; i < 10; i++)
        {
            System.out.print(i + " ");
        }
        System.out.println();
    }
}
```

Oppure

```
package thread2;

public class Thread2 extends Thread {
    public static void main(String[] args)
    {
        Thread2[] t = new Thread2[10];
        for(int i = 0; i < 10; i++)
        {
            t[i] = new Thread2();
            t[i].setName(Integer.toString(i));
            t[i].start();
        }
        System.out.println("Fine");
    }
    @Override
    public void run()
    {
        System.out.println("Thread ID " + this.getName());
        for(int i = 0; i < 10; i++)
        {
            System.out.print(i + " ");
        }
        System.out.println();
    }
}
```


Utilizzo Thread – secondo metodo

Un secondo metodo è quello di implementare l'interfaccia `Runnable` come in questo esempio

In alcuni casi non è possibile o utile estendere la classe `Thread` (ad es. se `Thread` è già un'estensione di un'altra classe o perché si vogliono avviare molteplici thread dentro la stessa istanza di un oggetto). In questo caso si usa l'interfaccia `Runnable`.

Vantaggio di questa soluzione: possibilità di sfruttare l'ereditarietà multipla di interfacce usando i `Thread`.

```
package prova.runnable;
```

```
public class ProvaRunnable implements Runnable {  
    public ProvaRunnable() { }  
    @Override  
    public void run() {  
        System.out.println("Hello world");  
    }  
  
    public static void main(String[] args) {  
        ProvaRunnable test= new ProvaRunnable();  
        Thread hello= new Thread(target:test);  
        hello.start();  
    }  
}
```

Metodo join

void join()

Waits for this thread to die.

void join(long millis)

Waits at most `millis` milliseconds for this thread to die.

void join(long millis, int nanos)

Waits at most `millis` milliseconds plus `nanos` nanoseconds for this thread to die.

Gestire le eccezioni

Quando si utilizzano i Thread è sempre utile gestire le eccezione con:

InterruptedException

Così se uno o più Thread non si concludono si rileva l'evento e si può gestire/comunicare quanto è avvenuto.

```
try {  
    threadPari.join( millis:1000);  
    threadDispari.join( millis:1000);  
} catch (InterruptedException ex) {  
    System.out.println( x: "Errore");  
}
```

Esempio: thread stampa numeri positivi e negativi

Esempio: Creare 2 Thread, uno per stampare i numeri positivi, l'altro per stampare i numeri negativi. Utilizzare il controllo delle eccezioni

Start

Join

Try catch

```
package provathread;
public class ProvaThread {
    public static void main(String[] args) {

        ThreadPositivi t1 = new ThreadPositivi ();
        ThreadNegativi t2 = new ThreadNegativi ();
        System.out.println("Avvio il primo Thread");
        t1.start();           //avvio il thread
        System.out.println("Avvio il secondo Thread");
        t2.start();           //avvio il thread

        try {
            t1.join();
            System.out.println("Terminato il primo Thread");
            t2.join();
            System.out.println("Terminato il secondo Thread");
        } catch (InterruptedException e)
        {
            System.out.println("Errore nel Thread");
        }
    }
}
```

Nel progetto creare 2 classi:

ThreadPositivi che stampa i numeri da 0 a 99

ThreadNegativi che stampa i numeri da -1 a -99

L'esecuzione è "casuale",
cioè i thread vanno in
parallelo e quindi non
esiste una sincronia di
esecuzione.

```
package provathread;

public class ThreadPositivi extends Thread {
    public ThreadPositivi () { }
    public void run() {
        for (int i=0;i<100;i++)
            System.out.println("N "+i);
    }
}
```

```
package provathread;

public class ThreadNegativi extends Thread {
    public ThreadNegativi () { }
    public void run() {
        for (int i=-1;i>-100;i--)
            System.out.println("N "+i);
    }
}
```

Esercizio

Scrivere un programma Java che avvia tre nuovi thread: il primo stampa 100 volte il messaggio "Ciao", il secondo "A" e il terzo "Tutti".

Il thread principale (main) deve assegnare un nome ("T1", "T2", "T3") ad ogni thread.

Ogni thread deve effettuare una pausa di un secondo tra un messaggio e il successivo e, oltre al messaggio, deve stampare il proprio nome

Risolvere il problema prima usando una sola classe che estende Thread e poi usando un Runnable

Esercizio

Scrivi un programma che generi un thread secondario che stampa ogni secondo un contatore del numero di secondi di esecuzione.

Il thread principale sospende l'esecuzione dopo aver letto un numero intero.

Stampare nel thread principale il numero di secondi del thread secondario.

Soluzione

```
package nb_thread11;

class CounterThread implements Runnable {
    private int secondiPassati;

    public void run() {
        secondiPassati = 0;
        while (!Thread.currentThread().isInterrupted()) {
            try {
                Thread.sleep(1000);
                secondiPassati++;
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
    }

    public int getSecondiPassati() {
        return secondiPassati;
    }
}
```

Scrivi un programma che generi un thread secondario che stampa ogni secondo un contatore del numero di secondi di esecuzione. Il thread principale sospende l'esecuzione dopo aver letto un numero intero. Stampare nel thread principale il numero di secondi del thread secondario.

Soluzione

```
package nb_thread11;
/*
1) Scrivi un programma che generi un thread secondario che stampa ogni secondo un contatore
del numero di secondi di esecuzione.
Il thread principale sospende l'esecuzione dopo aver letto un numero intero.
Stampare nel thread principale il numero di secondi del thread secondario.
*/
import java.util.Scanner;
public class Nb_Thread11 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        CounterThread counterThread = new CounterThread();
        Thread t = new Thread(counterThread);
        t.start();
        System.out.println("Inserisci un numero intero per fermare il thread: ");
        scanner.nextInt();
        scanner.close();
        t.interrupt();

        System.out.println("Il thread secondario ha eseguito per " + counterThread.getSecondiPassati() + " secondi");
    }
}
```

Esercizi

Scrivere un programma che sommi i numeri da 1 a 10.000. Usare 2 thread, uno somma i numeri da 1 a 5.000, l'altro i numeri da 5.001 a 10.000.

```
package thread4;

public class RunnableClass implements Runnable {

    private long start;
    private long end;
    private long result;

    RunnableClass() {
        start = 0;
        end = 0;
        result = 0;
    }

    RunnableClass(int start, int end) {
        this.start = start;
        this.end = end;
        this.result = 0;
    }

    @Override
    public void run() {
        for (long i = start; i <= end; i++) {
            result = result + i;
        }
    }

    public long getResult() {
        return result;
    }
}
```

Soluzione

Nell'interfaccia runnable ci sono 2 costruttori con e senza parametri e nel metodo run, la somma dei numeri tra start ed end. C'è anche un metodo per restituire la somma

Soluzione

Nel main si creano i due thread e si stampa il risultato

```
package thread4;
public class Thread3 {
    public static void main(String[] args) {
        RunnableClass r1 = new RunnableClass(0,5000);
        RunnableClass r2 = new RunnableClass(5001,10000);
        Thread t1 = new Thread(r1);
        Thread t2 = new Thread(r2);

        t1.start();
        t2.start();

        try {
            t1.join(500);
        } catch (InterruptedException ex) {
            System.out.println("Errore 1");
        }

        try {
            t2.join(500);
        } catch (InterruptedException ex) {
            System.out.println("Errore 2");
        }

        System.out.println("-----");
        long risultato1 = r1.getResult();
        long risultato2 = r2.getResult();
        long rTot = risultato1 + risultato2;
        System.out.println("Risultato: " + rTot);
    }
}
```

Esercizio

Dato un array di DIM (dimensione inserita dall'utente) elementi generati casualmente compresi tra 0 e 100, scrivi un programma parallelo che ricerchi un valore K all'interno dell'array e dichiari esso è presente almeno una volta nell'array.

SOLUZIONE

```
package thread5;

public class RunnableClass implements Runnable {

    private int[] array;
    private int k;
    private boolean isPresent;
    private int start;
    private int end;

    RunnableClass(int[] array, int k, int start, int end) {
        this.array = array;
        this.k = k;
        this.isPresent = false;
        this.start = start;
        this.end = end;
    }

    @Override
    public void run() {
        for (int i = start; i < end; i++) {
            if (array[i] == k) {
                isPresent = true;
                break;
            }
        }
    }

    public boolean getIsPresent() {
        return isPresent;
    }
}
```

SOLUZIONE

```
package thread5;
import java.util.Random;

public class Thread5 {

    final static private int DIM = 50000;

    public static void main(String[] args) {
        int[] array = new int[DIM];
        Random random = new Random();

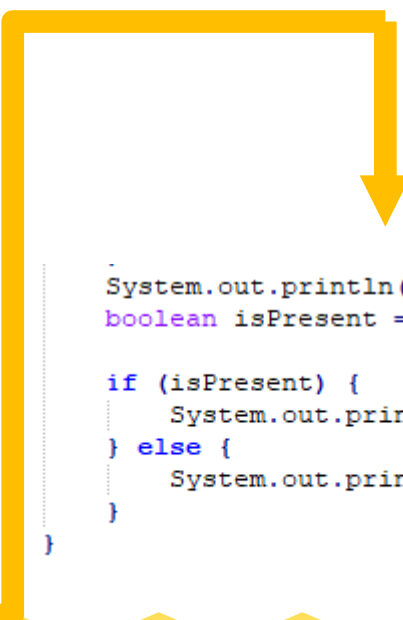
        for (int i = 0; i < DIM; i++) {
            array[i] = random.nextInt(100);
        }

        int k = -1;
        RunnableClass r1 = new RunnableClass(array, k, 0, DIM / 2 - 1);
        RunnableClass r2 = new RunnableClass(array, k, DIM / 2, DIM);
        Thread t1 = new Thread(r1);
        Thread t2 = new Thread(r2);

        t1.start();
        t2.start();

        try {
            t1.join(500);
        } catch (InterruptedException ex) {
            System.out.println("Errore 1");
        }

        try {
            t2.join(500);
        } catch (InterruptedException ex) {
            System.out.println("Errore 2");
        }
    }
}
```



```
System.out.println("-----");
boolean isPresent = r1.getIsPresent() || r2.getIsPresent();

if (isPresent) {
    System.out.println("C'è");
} else {
    System.out.println("non C'è");
}
```

Esercizio 13/9

- Scrivere un programma che calcoli con N thread differenti i valori fattoriali di N numeri naturali tra 1 e 10 generati casualmente e memorizzati in un vettore.

Esercizi

- 1) Progettare un programma che, una volta avviato, chieda in input due valori interi e generi due processi figli, il primo si occuperà di calcolare e visualizzare il prodotto dei due valori, il secondo la somma.
- 2) Scrivere un programma che, una volta avviato, generi due processi figli che a loro volta generino un proprio processo figlio.
Impostare opportunamente l'output in modo tale che sia comprensibile quali siano le relazioni di "parentela".
- 3) Si vogliono sommare tutti gli elementi di una matrice intera $N \times M$ con un insieme limitato N di thread (un thread per riga) che sommino i numeri delle righe in parallelo. Il thread principale deve sincronizzarsi sulla terminazione dei thread figli e stampare poi il risultato finale (la somma dei totali delle righe) a video.

Esercizi

Scrivere un programma che, una volta avviato generi un processo figlio. Il processo padre dovrà visualizzare l'alfabeto maiuscolo con un ritardo di 1 secondo (provate con valori diversi), mentre il processo figlio dovrà visualizzare l'alfabeto minuscolo. Per la visualizzazione degli alfabeti si usino dei cicli `for()` che visualizzano i caratteri ASCII delle lettere dell'alfabeto maiuscolo e minuscolo.

Metodi dei Thread

sleep sospende l'esecuzione del thread per i millisecondi indicati e genera una eccezione **InterruptedException** se fermato

```
Thread.sleep(200)
```

setName associa un nome in fase di creazione di un Thread

```
Thread.setName("MioThread")
```

interrupt() scatena la richiesta di interruzione per interrompere il thread

isInterrupted() restituisce true se il thread t è interrotto, false se è attivo

Interrupted() restituisce true se è in atto una richiesta di interruzione

Condivisione della memoria

Condivisione di un oggetto fra due thread:

```
int[ ] x = new int[ ] { 1,2,3,4,5,6,7,8 }; // oggetto  
condiviso
```

```
MioThread t1 = new MioThread( x );
```

```
MioThread t2 = new MioThread( x );
```

```
t1.start();
```

```
t2.start();
```

x è fisicamente lo stesso oggetto e non viene “copiato” nei due Thread.

Per creare una variabile condivisa tra più thread è sufficiente creare un oggetto prima della creazione dei thread e passarlo ai thread prima di avviarli (nel costruttore)

In questo modo ogni thread riceve un riferimento allo stesso oggetto (che quindi è presente una sola volta in memoria)

Accedendo all'oggetto, i thread potranno leggere/scrivere la **stessa area di memoria** (--> **memoria condivisa**)

ESERCIZIO

Simulare un allenatore che tramite comandi interattivi (corri, cammina, fermati e basta) letti dalla classe Scanner dà dei comandi ad un corridore che si sta allenando.

Creare una classe Corridore che estende Thread. Essa deve dichiarare i metodi: corri() che gli permetterà di simulare una corsa, cammina() che gli permetterà di simulare una camminata, fermati() che gli permetterà di simulare l'azione del fermarsi, e basta() che farà terminare l'allenamento (e il programma).

Corsa critica - race condition

Una **corsa critica** si sviluppa quando due (o più) thread tentano di modificare lo stesso oggetto nel momento in cui, mentre almeno un altro thread non ha terminato di eseguire le operazioni.

Un programma contiene una **race condition** quando il risultato finale dell'elaborazione (output) è soggetto a variazioni casuali poiché dipende dalla particolare sequenza temporale con cui alcune istruzioni concorrenti sono eseguite.

SOLUZIONE:

Una **sezione critica** è una porzione di codice del programma che non deve essere eseguita in modo concorrente da parte di più thread o processi.

Una **sezione critica** è una parte di codice che può essere eseguita da un solo thread o un processo contemporaneamente.

Synchronized

```
public class esempio{  
  
    public synchronized void metodo_1() {  
    }  
  
    public synchronized void metodo_2() {  
    }  
  
    public void metodo_3() {  
    }  
}
```

La parola chiave **synchronized** crea la sezione critica all'interno del metodo: tutto ciò che c'è all'interno viene visto come un'operazione atomica. In questo caso l'esecuzione di metodo 1 e metodo 2 è atomica mentre metodo 3 è libera.

Ogni metodo può essere reso atomico con `synchronized` in qualsiasi classe e/o ogni metodo non `synchronized` può chiamare oggetti `synchronized`.

Metodi wait e notify

PROBLEMA: due thread condividono una variabile intera. Il primo thread deve aspettare che il secondo ne modifichi il valore per poter andare avanti.

Per poter gestire questo problema si utilizzano **tre metodi** della classe Object

public void wait() throws InterruptedException mette il thread corrente in attesa. Come tutti i metodi bloccanti, wait è sensibile allo stato di interruzione del thread quindi **deve essere inserito in un try-catch**.

public void notify()

public void notifyAll()

La differenza tra notify e notifyAll è che il primo risveglia uno solo dei thread che sono potenzialmente in attesa della condizione, mentre notifyAll li sveglia tutti.

I semafori

Il semaforo si utilizza per la sincronizzazione di thread. Si può utilizzare sia per inviare segnali tra i thread che per proteggere una sezione critica di codice.

Quando si inizializza un semaforo è necessario indicare il numero di thread permessi cioè il numero di thread che possono acquisire il lock sul semaforo senza essere bloccati.

```
Semaphore semaphore = new Semaphore(N);
```

Semafori

- Il semaforo fornisce principalmente due metodi: **acquire()** (acquisisce una risorsa) e **release()** (rilascia la risorsa). Quando le N risorse sono esaurite il thread successivo che esegue `acquire()` sarà bloccato fino al prossimo `release()`.
- Quando si utilizza per proteggere una sezione critica di codice, il programma deve invocare il metodo `acquire()` prima dell'esecuzione di tale sezione ed il `release()` al termine di tale sezione.

```
Semaphore semaphore = new Semaphore(1);  
  
...  
//sezione critica  
semaphore.acquire();  
  
...  
...  
semaphore.release();
```

I metodi della classe semaforo

	<u>Semaphore</u> (int permits) Creates a Semaphore with the given number of permits and nonfair fairness setting.	Il numero di risorse è incluso nel costruttore.
void	<u>acquire</u> () Acquires a permit from this semaphore, blocking until all are available, or the thread is <u>interrupted</u> .	Acquire=down, eventualmente bloccante per il thread se il valore del contatore è zero e il thread che ha invocato la acquire resta fermo fintantoche un altro thread non chiama la release()
void	<u>release</u> () Releases a permit, returning it to the semaphore.	La release può sbloccare più processi in attesa eventualmente sullo stesso semaforo.
int	<u>availablePermits</u> () Returns the current number of permits available in this semaphore.	Ritorna il numero di risorse disponibili per il semaforo

Esempio semaforo

N persone desiderano entrare in una stanza. La stanza ha una capienza massima di C persone. Ogni persona, una volta entrata nella stanza, vi resta per T secondi, poi esce. Una persona che trova la stanza piena attende finchè non si libera un posto.

Scrivere un programma Java che simula la situazione appena descritta, modellando ciascuna persona come un Thread e utilizzando la classe Semaphore per regolare l'accesso.

```
*/  
public class Semaphore {  
  
    private int contatore;  
  
    public Semaphore(int i)  
    {  
        this.contatore=i;  
    }  
    public Semaphore()  
    {  
        this.contatore=1;  
    }  
  
    public synchronized void release()  
    {  
        contatore++;  
        if(contatore==1)  
            this.notify();  
    }  
  
    public synchronized void acquire() throws InterruptedException  
    {  
        if(contatore==0)  
            this.wait();  
        contatore--;  
    }  
}
```

ESERCIZIO

Un gruppo di 10 persone è all'osservatorio astronomico per ammirare il passaggio di una cometa sfruttando il potente telescopio messo a disposizione dalla struttura. Solo una persona alla volta può usare il telescopio, e i responsabili concedono solo tre minuti (secondi) a persona per completare l'osservazione. Non esiste una fila, i partecipanti accederanno al telescopio in maniera casuale. Ogni partecipante quindi attraverserà degli stati: lo stato "In attesa" quando starà aspettando il suo turno che dura un tempo indefinito, che dipende da quando inizierà il turno del partecipante; lo stato "Osservazione" quando starà osservando la cometa tramite il telescopio. Dura esattamente 3 minuti (secondi); lo stato "Finito" quando il turno è finito. Gli stati possono essere caratterizzati da delle stampe significative.

Esercizio parcheggio

```
package coda;
import java.util.*;

public class Auto extends Thread{
    private int contatore=0;
    private static int T = 5;
    Semaphore postiliberi;
    private double fatturato;

    public Auto(Semaphore P){
        this.postiliberi=P;
        fatturato=0;
    }

    public void run(){
        try{
            this.postiliberi.acquire();
        }
        catch(InterruptedException ex){
            System.out.println("ERRORE");
        }
        System.out.println("Il thread "+this.getName()+" e' entrato nel parcheggio");
        try{
            Random random = new Random();
            int tempo;
            tempo = random.nextInt(1901)+100;
            Thread.sleep(tempo);
            fatturato = fatturato + ((double) (tempo/1000))*(0.5));
        }
        catch(InterruptedException ex){
            System.out.println("ERRORE");
        }
        System.out.println("Il thread "+this.getName()+" esce dal parcheggio");
        this.postiliberi.release();
    }

    public double getFatturato(){
        return fatturato;
    }
}
```

Auto

Esercizio parcheggio

```
package coda;

public class Semaphore {
    private int counter; //quante sono le risorse disponibile nella sezione critica

    Semaphore(){
        this.counter=1;
    }

    Semaphore(int C){
        this.counter=C;
    }

    //acquire() prova ad occupare una risorsa di quelle disponibili
    //con la parola synchronized java gestisce la corsa critica
    public synchronized void acquire() throws InterruptedException{
        if(this.counter==0)
            this.wait();
        this.counter--;
    }

    public synchronized void release(){
        this.counter++;
        if(counter>0)
            this.notify(); //sblocco eventualmente il thread che voleva "entrare" ed ere stato bloccato prima
    }
}
```

Semaforo

Esercizio parcheggio

```
package coda;

import java.util.logging.Level;
import java.util.logging.Logger;

public class TestParcheggio {
    public static void main(String[] args) {
        int c = 100;
        int N = 1000;

        Semaphore S = new Semaphore(c);
        Auto T[] = new Auto[N];
        int sum=0;
        for(int i=0;i<N;i++){
            T[i] = new Auto(S);
            T[i].setName(Integer.toString(i+1));
            T[i].start();
        }
        for(int i=0;i<N;i++){
            try {
                T[i].join(50);
                sum+=T[i].getFatturato();
            } catch (InterruptedException ex) {
                Logger.getLogger(TestParcheggio.class.getName()).log(Level.SEVERE, null, ex);
            }
        }
        System.out.println("Il fatturato totale e' di "+sum+" euro");
    }
}
```

Main

Deadlock

Produttori-consumatori

Una situazione ricorrente nella programmazione concorrente consiste nel paradigma produttore-consumatore

Si tratta di due o più thread, divisi in due categorie:

- **I produttori (producer)** sono fonti di informazioni destinate ai consumatori
- **I consumatori (consumer)** devono elaborare le informazioni fornite dai produttori, non appena queste si rendono disponibili

Non è possibile prevedere quanto tempo impiega un produttore a produrre un'informazione né quanto impiega un consumatore ad elaborarla. E' perciò necessario prevedere uno o più buffer, che contengono le informazioni prodotte e non ancora consumate

